

Conditionals and code documentation

Reading

- [PY4E chapter 3 \(https://www.py4e.com/html3/03-conditional\)](https://www.py4e.com/html3/03-conditional)
- [PEP-8 \(https://peps.python.org/pep-0008/\)](https://peps.python.org/pep-0008/)
- [Google's Python styleguide \(https://google.github.io/styleguide/pyguide.html#38-comments-and-docstrings\)](https://google.github.io/styleguide/pyguide.html#38-comments-and-docstrings) section 3.8

Exercises

At this step you will be able to start working with a few of our [Python exercises \(https://people.dsv.su.se/~robe/lis020/exercises/\)](https://people.dsv.su.se/~robe/lis020/exercises/). Please do this, in addition to the PY4E exercises in chapter 3.

In addition, practice following the Python style guides and using type annotations while working on your exercises.

Conditionals

Recall the `make_past` function from the previous lecture:

```
In [1]: def make_past(stem: str) -> str:
        suffix: str = "ed"
        word: str = stem + suffix
        return word
```

In some cases, it works fine:

```
In [2]: make_past("lik")
```

```
Out[2]: 'liked'
```

In other cases, problems appear:

```
In [3]: make_past("program")
```

```
Out[3]: 'programed'
```

The problem here is that the **form of the past tense suffix depends on the stem**. If the stem ends with *m*, the suffix becomes *-med* rather than *-ed*. This is an instance of an extremely common situation in programming, where you want the program to do different things depending on some piece of information.

Conditionals solve this problem.

The most common conditional in Python is the `if / elif / else` statement. The simplest version of the statement has only a single `if`, followed by an indented block of code to be performed in case the condition after `if` is satisfied:

```
In [4]: def make_past(stem: str) -> str:
        suffix: str = "ed"
        if stem[-1] == "m":
            suffix = "med"
        word: str = stem + suffix
        return word
```

```
In [5]: make_past("lik")
```

```
Out[5]: 'liked'
```

```
In [6]: make_past("program")
```

```
Out[6]: 'programmed'
```

Now the code works as we want it to, both for stems with and without an *m* at the end.

The code `stem[-1]` gets the last character of the string in the variable called `stem`. The operator `==`, **not to be confused** with a single equals sign (`=`), produces the boolean value `True` if the values on the left and the right are equal.

Breaking down code

While the Python Visualizer is useful to follow a piece of code line by line, sometimes many things happen in a single line of code. It will become very important that you are able to break a piece of code into its smallest components, and understand what happens and in which order.

A useful way to test your own knowledge is to break down code into simpler code, in particular assigning a named variable to each value computed. With the above example, this could be done in the following way:

```
In [7]: def make_past(stem: str) -> str:
        suffix: str = "ed"
        last_letter: str = stem[-1]
        last_letter_is_m: bool = last_letter == "m"
        if last_letter_is_m:
            suffix = "med"
        word: str = stem + suffix
        return word
```

Note that after `if` you should have an **expression** that **evaluates to** `True` or `False`. This could be either literally `True` as in this silly example:

```
In [8]: if True:
        print("This always happens, obviously.")
```

This always happens, obviously.

Note here that the `else` part is missing. That is fine, only the `if` is mandatory.

Normally, there is something more interesting following `if`. One of the simplest cases is a plain variable, like this:

```
In [9]: word = "lik"
        last_letter = word[-1]

        last_letter_is_m = last_letter == "m"
        if last_letter_is_m:
            print("Last letter is M")
        else:
            print("Last letter is not M")
```

Last letter is not M

Or we could use a more complex **boolean expression**:

```
In [10]: if last_letter == "m":
          print("Last letter is M")
        else:
            print("Last letter is not M")
```

Last letter is not M

A common mistake is to confuse Python with a human, and try to tell it something like this:

```
In [11]: if last_letter == "m" or "n":
          print("Last letter is M or N")
        else:
            print("Last letter is not M or N")
```

Last letter is M or N

It runs, but wait a minute! We suddenly got a different result. Let's see what `last_letter` actually contains.

```
In [12]: last_letter
```

```
Out[12]: 'k'
```

Which is clearly not "m" or "n". What is happening here? Try to break down the code and see what goes wrong.

```
In [13]: last_letter_is_m = last_letter == "m"
        last_letter_is_m
```

```
Out[13]: False
```

So far so good. But now the next step is perhaps not what you expected:

```
In [14]: last_letter_is_m_or_n = last_letter_is_m or "n"
        last_letter_is_m_or_n
```

```
Out[14]: 'n'
```

Remember that in Python, the same operator can do different things on different types. For instance:

```
In [15]: 1+1
```

```
Out[15]: 2
```

```
In [16]: "1"+"1"
```

```
Out[16]: '11'
```

In this case, `last_letter_is_m` has the value `False`, so we are actually computing this:

```
In [17]: False or "n"
```

```
Out[17]: 'n'
```

This is a pretty weird definition of "or", but it's how Python does it and you just have to live with it. Now let us plug that "n" we got back into the if statement:

```
In [18]: if "n":  
        print("Last letter is M or N")  
        else:  
        print("Last letter is not M or N")
```

Last letter is M or N

This is because `if` forces any value to become a boolean before checking it. You can do the same thing manually to see what happens:

```
In [19]: bool("n")
```

```
Out[19]: True
```

So Python thinks "n" corresponds to `True`. In fact, Python thinks *all* strings except the empty string are `True`.

The point is this: **Python does what you say, not what you mean**. If you are in doubt, try to fall back on breaking the code up to do smaller but more straightforward operations. It is not immediately obvious that `last_letter == "m" or "n"` is wrong, but when you see `last_letter_is_m_or_n = last_letter_is_m or "n"` warning bells should be ringing.

Multiple cases

Often you have a bunch of cases you need to check. This can be done with `elif` and/or `else`. You can also put `if` statements inside other `if` statements:

```
In [20]: stem = input("Specify a verb stem: ")
if stem == "write":
    # Irregular stem 1
    word = "wrote"
elif stem == "speak":
    # Irregular stem 2
    word = "spoke"
elif stem == "swim":
    # Irregular stem 3
    word = "swam"
else:
    # No irregular stem, so we try the different regular affixes
    last_letter = stem[-1]
    if last_letter == "m":
        # Special regular pattern
        suffix = "med"
    else:
        # Most general regular pattern
        suffix = "ed"
    word = stem + suffix
print("The resulting word is:", word)
```

```
Specify a verb stem: swim
The resulting word is: swam
```

In the chain of `if`, `elif`, `elif`, ..., the conditions are checked one by one, and as soon as one is true the code inside that part is executed and the rest of the `if` chain is ignored. If none of the conditions is true, the code inside `else` (if there is any) is executed. If there is no `else` and no condition is true, nothing at all happens.

Exceptions

In practice, you often need to check if everything went according to expectations. For instance, that numbers input by the user really are numbers, or that files you want to open really exist, or that you have permission to read them, etc. This can become rather tedious, and Python has an **exception** mechanism for catching these (more or less) exceptional cases.

To give a simple example, consider the following code that asks for the age of your parents and computes their age difference:

```
In [22]: father_age = int(input("Age of father: "))
        mother_age = int(input("Age of mother: "))
        print("Age difference is", abs(father_age-mother_age))
```

Age of father: n/a

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-22-5055346a7ef3> in <module>
----> 1 father_age = int(input("Age of father: "))
      2 mother_age = int(input("Age of mother: "))
      3 print("Age difference is", abs(father_age-mother_age))

ValueError: invalid literal for int() with base 10: 'n/a'
```

If something is contrary to expectations, e.g. that we do not give a numeric value for the mother's age, we get an error message. This error message is in fact an **uncaught exception**, its name is `ValueError`. Uncaught exceptions generally crash our program, which is usually undesirable. With a `try / except` statement, you can **catch** the exception and do something more useful than crashing. For instance:

```
In [23]: try:
    father_age = int(input("Age of father: "))
    mother_age = int(input("Age of mother: "))
    print("Age difference is", abs(father_age-mother_age))
except ValueError:
    print("Ages must be integers!")
```

```
Age of father: n/a
Ages must be integers!
```

If anything causes a `ValueError` anywhere inside the `try` block (or, importantly, within any functions called inside the `try` block), then the `except ValueError` block is immediately executed. We also skip everything after the exception occurred in the `try` block, note that no "Age difference is ..." was printed.

A few well-placed `try / except` statements in your code can deal with a large number of problems. In theory you can use `if` statements to achieve the same thing, but that would become rather messy, especially if the problem occurs inside a function.

Note that you can also **raise exceptions**, using the `raise` keyword. This will immediately go to the next matching `except` clause, which could be in the same function or (more commonly) in the function that called the current one, or the function that called *that* one, etc. The convenient thing is that when you write the function with `raise`, you do not have to care about where the `except` will be, you can simply assume that whoever is going to use your function will put a `try / except` statement somewhere. If they fail to do so, the program will crash because of the uncaught exception.

```
In [24]: def get_first_letter(word: str) -> str:
    if len(word) == 0:
        raise ValueError("Word is empty")
    return word[0]

def get_first_letters(word1: str, word2: str) -> str:
    return get_first_letter(word1) + get_first_letter(word2)

def get_initials(name: str, surname: str):
    try:
        return get_first_letters(name, surname)
    except ValueError:
        return 'XX'
```

```
In [25]: get_initials("John", "Smith")
```

```
Out[25]: 'JS'
```

```
In [26]: get_initials("John", "")
```

```
Out[26]: 'XX'
```

Compare this to the case where the exception is *not* caught:

```
In [27]: get_first_letters("John", "")
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-27-4ecc9de6a56e> in <module>
----> 1 get_first_letters("John", "")

<ipython-input-24-1b0d557f5479> in get_first_letters(word1, word2)
      5
      6 def get_first_letters(word1: str, word2: str) -> str:
----> 7     return get_first_letter(word1) + get_first_letter(word2)
      8
      9 def get_initials(name: str, surname: str):

<ipython-input-24-1b0d557f5479> in get_first_letter(word)
      1 def get_first_letter(word: str) -> str:
      2     if len(word) == 0:
----> 3         raise ValueError("Word is empty")
      4     return word[0]
      5
```

```
ValueError: Word is empty
```

Code documentation

When writing a program, it is easy to produce "write-only code". There is a simple psychological reason for this: when working on a piece of code, you necessarily build a mental model of how the code is supposed to work. This is mainly stored in your short-term memory, and you typically revise this model frequently as you modify the code, discover problems, rearrange the structure of your solution, and so on.

After this, one of two things can happen:

1. You quickly move on to other tasks.
2. You spend time writing down an explanation of how your code works.

This is a trade-off. Option (1) makes sense if you are in a hurry and will not touch this particular piece of code again. What will inevitably happen is that your short-term memory fades, and you no longer understand the code (of course, other people will never have understood the code to begin with). This means that modifying the code later will be very difficult, because reading complex code and understanding how it works is **very difficult**.

Choosing option (2) will mean that you invest time now, in order to save time for yourself or others later. This is often a good idea, and you definitely need to know how to do it.

What needs to be documented?

There are two basic reasons that someone (including yourself) would want to (re)learn details about your code.

1. Learning how to **interface** your code from other code. This includes for instance which functions exist, what they do, which arguments are valid and what the return values will look like.
2. Understanding the fine details of your code in order to **modify** it.

Python has different types of solutions for these use cases. The main mechanism for documenting the **interface** of a program is through **docstrings**. Syntactically, these are simply strings that you insert into a Python program, but the text contained in these strings is ruled by conventions that may be enforced by external tools. Following at least one such convention is essential in order to ensure consistency. In this course we will not enforce any particular convention, but in our examples we generally follow the format recommended by the Google styleguide (see the reading list above). Docstrings are intended to be used by external tools and automatically summarized into documentation webpages, and much of the documentation you read online about Python is actually compiled from docstrings from a variety of Python code files.

For detailed comments, Python uses the `#` symbol for in-line comments. These are intended for the person trying to understand the details of a particular piece of code, typically in order to modify it. This type of documentation is less formalized than docstrings, and leaves you with a lot of freedom (and responsibility). Use comments to explain things that are not obvious to a *proficient Python programmer* reading your code. A typical beginner's mistake is to write comments like:

```
In [28]: # Compute 64 times 2 and store it in the variable x
x = 64 * 2
```

A proficient programmer already knows what the code does, but not the *purpose* of doing this. Something like this would make more sense:

```
In [29]: # Let the initial value of x (the total cost) be twice the cost of a pizza (64)
x = 64 * 2
```

Of course, an even better solution in this case would have been to use better variable names. In practice, code is made readable by combining good naming habits with in-line comments:

```
In [30]: pizza_cost = 64
# Initial value for the total cost, will be updated later
total_cost = 2 * pizza_cost
```

Putting it all together

To summarize the above, well-documented code typically combines a range of methods:

- Docstrings for functions, classes and modules
- Code comments with `#`
- Transparent names of variables, functions, arguments, classes and modules
- Type annotations

We can illustrate by showing the progress of one of the examples given earlier, as we add these layers of documentation. We start with the kind of code you should **not** be producing:

```
In [31]: def inflect(s):  
        x = "ed"  
        if s[-1] == "m":  
            x = "med"  
        y = s + x  
        return y
```

Of course, with such a simple function it is not very much work to read through the code and try to guess what the purpose is. In reality, functions may consist of tens of lines of complex code, where doing that would be a very tiring exercise. Let us start with adding more sensible names:

```
In [32]: def make_past(stem):  
        suffix = "ed"  
        if stem[-1] == "m":  
            suffix = "med"  
        word = stem + suffix  
        return word
```

This is already better. Adding **type annotations makes things a bit more explicit**, although it is already pretty obvious that all variables are strings, so the overall readability is not improved much (but remember that type annotations will also be useful for automated bug checking).

```
In [33]: def make_past(stem: str) -> str:  
        suffix: str = "ed"  
        if stem[-1] == "m":  
            suffix = "med"  
        word: str = stem + suffix  
        return word
```

Finally, let us add a **docstring (Google format)** and some in-line comments:

```
In [34]: def make_past(stem: str) -> str:
        """Generate the past tense given an English verb stem.

        This function only works correctly for a limited subset of regular verbs, either
        containing a simple -ed suffix or those that have a stem ending with -m that needs
        to be doubled before the -ed suffix is added.

        Args:
            stem: A string representing an English word stem.

        Returns:
            A string representing the past tense of the stem.
        """
        # Default suffix is -ed
        suffix: str = "ed"
        # Stems ending with "m" use -med
        if stem[-1] == "m":
            suffix = "med"
        word: str = stem + suffix
        return word
```

If we ask you to produce well-documented code, we mean something like this.

Note that to *use* the function, you only really need to look at the docstring. The rest of the documentation is meant for those modifying the function, for instance if you want to support a larger set of verbs at some point. There is no need to comment every line, unless the code is unusually complex and difficult to understand.

For simple functions that are only used internally in your code, documentation at this level may be a bit too much. If I were to write the above function as a part of my own program, not to be used by other people's code, something like this might be appropriate given the simplicity of the function:

```
In [35]: def make_past(stem: str) -> str:
        """Generate the past tense given a regular English verb stem."""
        suffix = "ed"
        if stem[-1] == "m":
            suffix = "med"
        word = stem + suffix
        return word
```

Note that the most important parts are there: the one-line summary of the docstring as well as the type annotations for the function itself (but not inside the function).

There are no absolute universal rules regarding documentation, although many organizations impose such rules on their employees. Try to get a feeling for what types of documentation are available and when they are useful, then use your judgement.